

SmartAdmi

Intelligent College Admission & Verification System

An AI-Driven Full-Stack Application for Automated Certificate OCR, Real-Time Course Recommendations, and Fraud Prevention Analysis.

PROJECT DOCUMENTATION & ARCHITECTURE HANDBOOK

Author: Google AI Studio Build Team

Version: Production Release 1.0.0 (TypeScript)

AI Platform: Google Gemini Developer Models (@google/genai)

Core Stack: React (v19) + Express Server + Firebase Realtime Firestore

Generated: July 2026 i18n AI Studio Verified Sandbox

1. Executive Summary & Objectives

Traditional college admission pipelines suffer from high manual overhead, slow operational turnarounds, and elevated vulnerability to document manipulation, transcript fraud, or clerical entry errors. SmartAdmi directly addresses these systemic bottlenecks by introducing a secure, fully automated, and intelligent admission ecosystem.

The system streamlines student registration and leverages cutting-edge artificial intelligence to cross-verify physical credentials, automatically audit academic records, and generate personalized course recommendations, establishing a reliable, frictionless experience for both students and universities.

Core Functional Goals

- **Automate Document Audits: OCR Extraction** Instantly parse text and grades from uploaded certificates, completely eliminating manual transcription.
- **Mitigate Credential Fraud: Authenticity Scoring** Evaluate user inputs against visual documents to yield an automated confidence coefficient.
- **Personalized Counseling: Intelligent Recommendations** Assess applicant metrics and career interests to offer matching engineering branches.
- **Interactive Guidance: Admission Companion Chatbot** Address applicant questions on enrollment timelines, requirements, and curricula 24/7.

2. System Architecture & Workflows

SmartAdmi is built upon a distributed full-stack architecture. The user interface, constructed in React, coordinates with an Express.js backend API that acts as a secure proxy to external services, isolating API secrets and preventing client-side leaks.

SYSTEM WORKFLOW SCHEMATIC

```
[Student Registration & Login] (Firebase Auth)
                                !iL½
[Submit Marks & Certificate Upload] (React Hook Form + Base64 Compressor)
                                !iL½
[Gemini Secure AI OCR, Comparison & Course Engine] (Express API Proxy)
                                !iL½
[Real-time Board Decision & Automated Status Notify] (Firestore + Nodemailer)
```

3. Detailed Technology Stack

The system utilizes industry-grade tools across every tier of execution to guarantee speed, modularity, visual luxury, and rigid security policies.

Core Runtime & Build Systems

The framework engine supporting compile-time validation and high-speed serving.

- **Node.js (v18/v20/v22):** Serves as the server runtime and hosting engine for our Express REST api.
- **TypeScript (v5.8):** Enforces strong type interfaces, static verification, and strict compile-time contracts.
- **Vite (v6.2):** Powers hot module optimization and bundles light assets for production assets.
- **esbuild (v0.28):** Bundles server TS code into a clean CommonJS file (dist/server.cjs) bypassing Node relative import errors.

Database & Intelligent AI Tiers

The databases holding user state and the intelligence layers parsing academic files.

- **Firebase Client SDK:** Allows instant database subscriptions and pushes application documents from the browser.
- **Firebase Admin SDK:** Runs elevated, server-authorized commands like duplicate checks on the Express backend.
- **Google Gemini API:** Leverages cutting-edge models via `@google/genai` to perform multimodal OCR, fraud matching, and chat.
- **Nodemailer:** Sends instant HTML mail notifications to students when administration boards review their application status.

Client UI & Experience Layer

A luxurious, responsive dashboard styled with deep focus on accessibility and motion.

- **Tailwind CSS (v4.1):** A utility styling engine delivering layout responsive grids across desktop and mobile devices.
- **Motion:** Produces visual rhythmic animations, staggered page entrances, and smooth route transitions.
- **Lucide React Icons:** Serves clean, scalable SVG symbols supporting interactive dashboards and application status pipelines.

4. Essential Source Code Architecture

The codebase is divided into modular, decoupled scripts mapping frontend rendering components from core security API processes.

Key File Map & Responsibility Matrix

- **src/pages/student/AdmissionForm.tsx: Admission Pipeline** A multi-step, responsive form. Collects candidate details, performs file compression to prevent database storage over-bloat, and triggers secure verification API requests.
- **src/pages/student/Status.tsx: Interactive Timeline** Renders a real-time status pipeline showing whether the student's credentials have cleared OCR and are currently pending board approval.
- **src/pages/admin/Dashboard.tsx: Admin Control Panel** Displays application metrics (average authenticity score, total submissions, engineering course trends) and handles manual approvals/rejections.
- **src/lib/gemini.ts: AI Intelligence client** Wraps the Google GenAI SDK. Extracts structured JSON for course recommendations and conducts fraud evaluation with structured fallback schemas.
- **server.ts: Express API Layer & Vite Server** Enforces server-side authentication proxy endpoints, handles CORS, implements file listeners, and securely compiles Vite assets.
- **src/lib/mlFraudService.ts: Heuristic Fraud Model** Acts as an offline-first backup utilizing a Naive Bayes classifier when network connectivity is lost or Firebase Firestore is unreachable.

5. Security Policies & Offline Resilience

To maintain high integrity in demanding environments, SmartAdmi incorporates extensive security protocols and intelligent, graceful offline fallbacks:

- **Secure API Key Shielding: Express Gateway** The Gemini API key is hosted completely server-side in the Node environment. No client-side code can access the key, ensuring zero key exposure.
- **Strict Firestore Rules: Document Access Guard** Rules ensure students can only create or view their own application file, while admin roles are explicitly authorized via custom claims lists.
- **Fail-Safe AI Recommendation: Dual-Execution Engine** If the Google Gemini service hits quotas, network disruptions, or has key issues, the Express backend automatically runs a mathematical fallback model matching the student's metrics to historical academic brackets seamlessly.
- **Offline Local Storage Cache: Firestore Offline Persistence** Active local caching allows students to draft and view their current progress even during transient network disconnects,

syncing changes as soon as connectivity resumes.

6. Local Development & Deployment Guide

SmartAdmi is configured to run instantly out-of-the-box on your computer. Follow these steps to configure your own free private database environment:

Step-by-Step System Bootstrap

- **1. Extract Workspace Archive:** Download the repository ZIP from AI Studio settings and extract it to a local folder.
- **2. Install Software Prerequisites:** Download Node.js (LTS version) and Visual Studio Code (VS Code) for your platform.
- **3. Establish Database: Firebase Console** Go to console.firebase.google.com. Create a project named 'SmartAdmi', enable Email/Password Authentication, and activate Firestore in test mode.
- **4. Configure Variables: VS Code .env** Replace the project API keys inside `firebase-applet-config.json` and create a root `.env` file containing your `GEMINI_API_KEY`.
- **5. Install Dependencies & Boot:** `npm install` Run 'npm install' in the VS Code terminal, followed by 'npm run dev' to boot the application on `http://localhost:3000`.

7. Future Roadmap Improvements

To scale SmartAdmi to support tier-1 academic institutions globally, the following features are planned for subsequent core rollouts:

- **Government Database Verification: DigiLocker Integration** Hook into sovereign academic database APIs to automatically cross-match certifications against cryptographic official registries.
- **Visual OCR Model Matching: Multimodal CNN Filters** Deploy custom vision classifiers to automatically identify document types (e.g. distinguishing a driver's license from a school marksheet) before extracting grades.
- **Predictive Student Analytics: Deep Learning Forecast** Build models predicting graduation success or academic performance in specific engineering branches based on historical multi-year records.
- **Sovereign Multi-Tenant Hosting: Kubernetes Deployment** Deploy stateless container clusters with Cloud SQL databases to process millions of parallel admissions securely.